

C#/.NET Teknik Değerlendirme

Yüksek Hacimli Event İşleme Sistemi

İş senaryosu

Bir finansal işlem platformu, farklı iş ortaklarından yüksek hacimde müşteri işlem event'leri almaktadır. Trafik zaman zaman kısa süreli sıçramalar göstermektedir. Mesajlaşma altyapısı at-least-once delivery garantisi verdiği için aynı event birden fazla kez ulaşabilir. Servis veya veri tabanı geçici olarak erişilemez olduğunda event'lerin kaybolmaması ve sistemin kontrolsüz biçimde kaynak tüketmemesi gerekir.

Göreviniz, event'leri kabul eden ve müşteri bazında günlük özetler üreten küçük bir mikroservis sistemi geliştirmektir.

Beklenen mimari

Çözüm en az iki ayrı deploy edilebilir bileşenden oluşmalıdır:

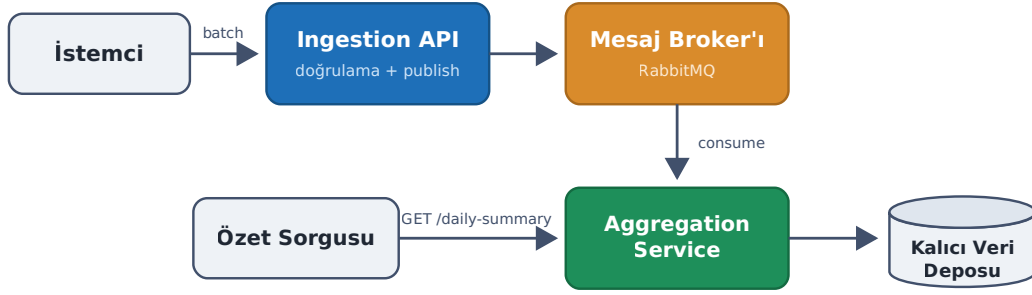
Ingestion API

- HTTP üzerinden event batch'leri kabul eder.
- Girdiyi doğrular.
- Kabul edilen event'leri mesaj broker'ına gönderir.

Aggregation Service

- Broker'dan event'leri tüketir.
- Event'leri idempotent biçimde işler.
- Günlük müşteri özetlerini kalıcı veri deposunda tutar.
- Oluşturulan özet sorgulamak için HTTP endpoint'i sunar.

Mesaj broker'ı olarak **RabbitMQ** kullanmanızı bekliyoruz. Kafka tercih edilirse çözüm çalıştırılabilir olmalı ve tercih nedeni README'de açıklanmalıdır. Kalıcı veri deposu olarak **PostgreSQL** önerilir; farklı bir tercih yapılabilir fakat gerekçesi belirtilmelidir.



Şekil 1 — Beklenen veri ve mesaj akışı

Event sözleşmesi

Her event aşağıdaki alanları içermelidir:

```

{
  "eventId": "5e35fcf8-bf9d-42e9-bdb2-d5c33c64eb52",
  "customerId": "customer-123",
  "type": "Credit",
  "amount": 125.50,
  "currency": "TRY",
  "occurredAt": "2026-08-06T10:15:30Z"
}
  
```

Alan	Kural
eventId	Geçerli bir UUID olmalıdır.
customerId	Boş olamaz ve en fazla 100 karakter olabilir.
type	Credit veya Debit olmalıdır.
amount	Sıfırdan büyük olmalıdır. Parasal hesaplamalarda decimal kullanılmalıdır.
currency	Üç harfli büyük karakterli para birimi kodu olmalıdır.
occurredAt	Zaman dilimi bilgisi içermeli ve günlük grupta için UTC'ye çevrilmelidir.

Duplicate davranışı: Aynı eventId birden fazla kez işlense bile özet yalnızca bir kez değişmelidir. Aynı eventId farklı içerikle gelirse özet ikinci kez değiştirilmemelidir; seçilen davranış loglanmalı ve README'de açıklanmalıdır.

Zorunlu fonksiyonel gereksinimler

1. Event kabulü

POST /api/v1/events/batch

- Bir istekte en fazla **1.000 event** kabul edilmelidir.
- Batch içindeki herhangi bir event geçersizse tüm istek reddedilmeli ve anlaşılır validation hataları dönmelidir.
- Broker event'leri kabul ettiğinde API **202 Accepted** dönmelidir.
- Başarılı cevap en az `batchId` ve `acceptedCount` alanlarını içermelidir.
- Broker'a erişilemiyorsa başarı cevabı dönlmemelidir.

Örnek cevap:

```
{
  "batchId": "80fc1f6e-b983-4b9e-b7d8-5e58c0435e16",
  "acceptedCount": 2
}
```

Bir batch publish edilirken kısmi hata oluşabileceğini dikkate alın. Seçtiğiniz davranışı ve istemcinin retry yapması durumundaki sonucu README'de açıklayın.

2. Günlük özet üretimi

Özetler aşağıdaki anahtara göre tutulmalıdır: `customerId` + `occurredAt` değerinin UTC günü + `currency`

Her özet en az şu bilgileri içermelidir:

- Toplam Credit tutarı
- Toplam Debit tutarı
- Net tutar: Credit – Debit
- İşlenen benzersiz event sayısı
- Son güncelleme zamanı

3. Özet sorgulama

GET /api/v1/customers/{customerId}/daily-summary?date=2026-08-06¤cy=TRY

- Kayıt bulunduğunda **200 OK** ve güncel özet dönmelidir.
- Kayıt bulunamadığında **404 Not Found** dönmelidir.
- Geçersiz tarih veya para birimi için **400 Bad Request** dönmelidir.

4. Idempotency ve tutarlılık

- Duplicate event'ler toplamaları veya event sayısını ikinci kez değiştirmemelidir.
- Idempotency yalnızca process belleğine dayanmamalıdır.
- Bir event'in işlendi olarak kaydedilmesi ile özeti güncellenmesi tutarlı olmalıdır.
- Birden fazla consumer aynı event'i eşzamanlı işlemeye çalıştığında sonuç değişmemelidir.

5. Retry ve dead-letter

- Geçici hatalarda kontrollü retry uygulanmalıdır.
- Retry sayısı ve bekleme davranışı konfigüre edilebilir olmalıdır.
- Sonsuz ve beklemesiz requeue döngüsü oluşmamalıdır.
- Belirlenen deneme sayısından sonra işlenemeyen mesajlar dead-letter queue'ya yönlendirilmelidir.
- Geçersiz veya deserialize edilemeyen mesajlar sistemi durdurmamalıdır.

6. Backpressure ve kaynak kullanımı

- Consumer concurrency ve broker prefetch/buffer sınırları konfigüre edilebilir olmalıdır.
- Sınırsız `Task.Run` , sınırsız `Task.WhenAll` veya sınırsız bellek kuyruğu kullanılmamalıdır.
- Servis kapanırken devam eden işler kontrollü biçimde sonlandırılmalı ve cancellation desteklenmelidir.

Gözlemlenebilirlik

Çözüm en az aşağıdakileri sağlamalıdır:

- Yapılandırılmış loglar
- Loglarda uygun yerlerde `batchId` , `eventId` ve `customerId`
- Kabul edilen, işlenen, duplicate ve başarısız event sayıları için metrikler
- Event işleme süresi için bir metric veya histogram
- Servisler için liveness ve readiness health endpoint'leri
- Hassas olmayan konfigürasyonların environment variable veya configuration dosyası üzerinden değiştirilebilmesi

Belirli bir monitoring ürünü veya dashboard geliştirilmesi beklenmemektedir.

Yük senaryosu

Çözümle birlikte en az **100.000 sentetik event** gönderebilen basit bir load generator, script veya test aracı sağlayın.

- Event'lerin en az **%10'u duplicate** olmalıdır.
- En az **100 farklı müşteri** bulunmalıdır.
- Credit ve Debit event'leri birlikte üretilmelidir.
- Event'ler batch'ler hâlinde gönderilmelidir.
- Sonuçta en az toplam süre, kabul edilen event sayısı, işlenen benzersiz event sayısı ve hata sayısı raporlanmalıdır.

Donanımına bağlı sabit bir requests-per-second hedefi yoktur. README'de test ortamınızı, gözlemlediğiniz sonucu ve ilk darboğazın nerede oluşacağını düşündüğünüzü kısaca açıklayın.

Test beklentileri

En az aşağıdaki davranışların otomatik testlerle gösterilmesini bekliyoruz:

- Doğru günlük aggregate hesaplaması

- Aynı event'in tekrar veya eşzamanlı gelmesi durumunda idempotency
- Geçersiz batch'in reddedilmesi
- Tüketim sırasında oluşan bir hatanın retry/DLQ davranışı

Unit ve integration test dengesini siz belirleyebilirsiniz. Testlerin nasıl çalıştırılacağı tek bir komutla veya açık adımlarla belgelenmelidir.

Teknik kısıtlar

- C# ve **.NET 8** veya daha yeni bir sürüm kullanılmalıdır.
- Servisler ayrı process/container olarak çalışmalıdır.
- Çözüm yerel ortamda `docker compose up --build` veya eşdeğer, açıkça belgelenmiş tek bir akışla ayağa kalkmalıdır.
- Veri şeması otomatik migration veya belgelenmiş kurulum adımıyla oluşturulmalıdır.
- Secret veya makineye özgü dosya repository'ye eklenmemelidir.
- Harici NuGet paketleri kullanılabilir; önemli seçimler açıklanmalıdır.

Teslimat

Repository aşağıdakileri içermelidir:

- Çalışan kaynak kod
- Docker Compose ve gerekli container tanımları
- Otomatik testler
- Load generator veya load test script'i
- Örnek istekler veya .http/Postman koleksiyonu
- Kapsamlı bir README

README içinde şu başlıklar bulunmalıdır:

- Sistemi çalıştırma ve test etme adımları
- Mimari ve servis sorumlulukları
- Veri ve mesaj akışı
- Idempotency ve transaction yaklaşımı
- Retry, DLQ ve backpressure yaklaşımı
- Yük testi sonucu ve gözlemler
- Bilinen eksikler ve daha fazla sürede yapılacaklar
- Önemli teknik kararlar ve trade-off'lar
- Kullanılan AI veya kod üretim araçları ve hangi amaçlarla kullanıldıkları

Kapsam dışı

Aşağıdakilerin geliştirilmesi beklenmemektedir:

- Authentication ve authorization
- Kullanıcı arayüzü
- Kubernetes manifestleri
- Cloud deployment
- Multi-region mimari
- Tam kapsamlı CI/CD pipeline
- Production seviyesinde dashboard ve alarm tanımları
- Her edge case'in eksiksiz çözümü

İsteğe bağlı geliştirmeler

Zorunlu kapsam tamamlandıysa aşağıdakilerden uygun gördüklerinizi ekleyebilirsiniz:

- OpenTelemetry distributed tracing
- Testcontainers ile integration testleri
- Publisher confirm veya outbox yaklaşımının daha güçlü uygulanması
- Şema versiyonlama stratejisi
- Rate limiting
- Performans profili ve allocation analizi
- Consumer ölçekleme veya partitioning yaklaşımının belgelenmesi

Bonus özellikler, zorunlu kapsamın doğruluğundan ve sadeliğinden daha değerli değildir.

Araç kullanımı

Dokümantasyon, internet, IDE yardım özellikleri ve AI tabanlı araçlar kullanılabilir. Teslim ettiğiniz kodun tamamını anlayabilmeniz, teknik kararları açıklayabilmeniz ve kullandığınız araçları README'de belirtmeniz beklenir.